

Библиотеки управления оборудованием

Содержание

1	Вводная часть	3
2	Модуль монохроматора	3
2.1	Создание объекта	3
2.2	Подключение и отключение	3
2.3	Управление длиной волны	3
2.4	Управление решётками	4
2.5	Управление щелями	4
2.6	Управление зеркалами	4
2.7	Управление фильтрами	5
2.8	Управление затвором	5
2.9	Получение информации	5
2.10	Константы модуля	6
3	Модуль лазера	6
3.1	Создание объекта	6
3.2	Подключение и отключение	6
3.3	Управление длиной волны	6
3.4	Управление двигателем	6
3.5	Управление затвором	7
3.6	Сброс устройства	7
3.7	Получение информации	7
3.8	Константы модуля	7
4	Модуль осциллографа	7
4.1	Создание объекта	7
4.2	Подключение и отключение	8
4.3	Управление каналами	8
4.4	Управление временной развёрткой	8
4.5	Управление триггером	9
4.6	Управление захватом	9
4.7	Захват данных	9
4.8	Измерения сигнала	9
4.9	Сохранение данных	10
4.10	Получение информации	10
4.11	Константы модуля	10

5	Модуль энергометра	11
5.1	Создание объекта	11
5.2	Подключение и отключение	11
5.3	Измерения параметров	11
5.4	Управление шкалой	11
5.5	Настройка длины волны	12
5.6	Дополнительные настройки	12
5.7	Получение информации	12
5.8	Константы модуля	13
6	Практические примеры	13
6.1	Подключение всех устройств	13
6.2	Проведение измерения	13
6.3	Сохранение результатов	14
6.4	Завершение работы	14

1 Вводная часть

Библиотеки предназначены для управления лабораторным оборудованием через язык Python. Каждая библиотека предоставляет удобный интерфейс для работы с конкретным типом устройства. Все библиотеки следуют единому стилю:

- `connect()` — устанавливает соединение с устройством.
- `disconnect()` — закрывает соединение.
- `is_connected()` — проверяет наличие соединения.
- `get_device_information()` — возвращает информацию об устройстве.
- `get_all_settings()` — возвращает все текущие настройки.

2 Модуль монохроматора

Управление монохроматором через комплект средств разработки.

2.1 Создание объекта

```
1 from chromator import Chromator
2
3 # Обычное создание
4 chromator = Chromator()
5
6 # С указанием пути к комплекту средств разработки
7 chromator = Chromator(sdk="C:/path/to/sdk")
```

2.2 Подключение и отключение

- `connect()` — подключается к монохроматору, возвращает `True` при успехе.
- `disconnect()` — отключается от монохроматора.
- `is_connected()` — проверяет наличие подключения, возвращает `True` при наличии соединения.
- `get_last_error()` — возвращает текст последней ошибки.

2.3 Управление длиной волны

- `set_wavelength(wavelength)` — устанавливает длину волны в нанометрах.
- `set_wavelength_async(wavelength)` — асинхронно устанавливает длину волны в нанометрах.
- `get_wavelength()` — возвращает текущую длину волны в нанометрах.
- `is_valid_wavelength(wavelength)` — проверяет возможность установки указанной длины волны.

- `wait_for_wavelength_stable(wavelength, tolerance, timeout)` — ожидает стабилизации длины волны с заданным допуском в нанометрах и таймаутом в секундах.

2.4 Управление решётками

- `get_grating_count()` — возвращает количество доступных решёток.
- `get_active_grating()` — возвращает индекс активной решётки.
- `set_active_grating(index)` — устанавливает активную решётку по индексу.
- `set_active_grating_async(index)` — асинхронно устанавливает активную решётку по индексу.
- `get_grating_parameters(index)` — возвращает параметры решётки: количество штрихов на миллиметр, минимальную длину волны, максимальную длину волны и угол блеска.
- `reset_grating()` — сбрасывает решётку в реперное положение.
- `reset_grating_async()` — асинхронно сбрасывает решётку в реперное положение.
- `reset_and_set_wavelength(wavelength)` — сбрасывает решётку и устанавливает указанную длину волны.
- `get_dispersion()` — возвращает дисперсию активной решётки.

2.5 Управление щелями

- `get_slit_count()` — возвращает количество щелей.
- `get_slit_name(index)` — возвращает название щели по её индексу.
- `set_slit_width(index, width, reset)` — устанавливает ширину щели в микрометрах с возможностью сброса перед установкой.
- `set_slit_width_async(index, width, reset)` — асинхронно устанавливает ширину щели в микрометрах с возможностью сброса перед установкой.
- `get_slit_width(index)` — возвращает текущую ширину щели в микрометрах.

2.6 Управление зеркалами

- `get_mirror_count()` — возвращает количество зеркал.
- `get_mirror_name(index)` — возвращает название зеркала по его индексу.
- `get_mirror_state_count(index)` — возвращает количество состояний зеркала по его индексу.
- `get_mirror_state_name(mirror, state)` — возвращает название состояния зеркала по индексам зеркала и состояния.

- `get_mirror_state(index)` — возвращает текущее состояние зеркала по его индексу.
- `set_mirror_state(mirror, state)` — устанавливает состояние зеркала по индексам зеркала и состояния.
- `set_mirror_state_async(mirror, state)` — асинхронно устанавливает состояние зеркала по индексам зеркала и состояния.

2.7 Управление фильтрами

- `get_filter_count()` — возвращает количество фильтров.
- `get_filter_name(index)` — возвращает название фильтра по его индексу.
- `get_filter_state_count(index)` — возвращает количество состояний фильтра по его индексу.
- `get_filter_state_name(filter, state)` — возвращает название состояния фильтра по индексам фильтра и состояния.
- `get_filter_bandwidth(filter, state)` — возвращает полосу пропускания фильтра по индексам фильтра и состояния.
- `get_filter_state(index)` — возвращает текущее состояние фильтра по его индексу.
- `set_filter_state(filter, state)` — устанавливает состояние фильтра по индексам фильтра и состояния.
- `set_filter_state_async(filter, state)` — асинхронно устанавливает состояние фильтра по индексам фильтра и состояния.

2.8 Управление затвором

- `get_shutter_count()` — возвращает количество затворов.
- `get_shutter_name(index)` — возвращает название затвора по его индексу.
- `shutter_open(index)` — открывает затвор по его индексу.
- `shutter_open_async(index)` — асинхронно открывает затвор по его индексу.
- `shutter_close(index)` — закрывает затвор по его индексу.
- `shutter_close_async(index)` — асинхронно закрывает затвор по его индексу.
- `get_shutter_state(index)` — возвращает состояние затвора по его индексу.

2.9 Получение информации

- `get_instrument_name()` — возвращает название прибора.
- `get_instrument_serial()` — возвращает серийный номер прибора.
- `get_device_information()` — возвращает словарь с информацией об устройстве.
- `get_all_settings()` — возвращает словарь со всеми текущими настройками.

2.10 Константы модуля

- `STATUS_READY = 1` — устройство готово к выполнению команд.
- `STATUS_BUSY = 2` — устройство выполняет команду.
- `STATUS_ERROR = -1` — произошла ошибка.
- `SHUTTER_CLOSED = 0` — затвор закрыт.
- `SHUTTER_OPENED = 1` — затвор открыт.
- `SHUTTER_MOVING = 2` — затвор находится в движении.

3 Модуль лазера

Управление источником лазерного излучения через последовательный порт.

3.1 Создание объекта

```
1 from laser_source import LaserSource
2
3 laser = LaserSource()
```

3.2 Подключение и отключение

- `connect(port, baudrate)` — подключается к лазеру через указанный порт и скорость, выполняет автоматический поиск если порт не указан.
- `disconnect()` — отключается от лазера.
- `is_connected()` — проверяет наличие подключения, возвращает `True` при наличии соединения.

3.3 Управление длиной волны

- `set_wavelength(wavelength)` — устанавливает длину волны в нанометрах.
- `get_wavelength()` — возвращает текущую длину волны в нанометрах.
- `wait_for_wavelength_stable(wavelength, tolerance, timeout)` — ожидает стабилизации длины волны с заданным допуском в нанометрах и таймаутом в секундах.

3.4 Управление двигателем

- `get_position(motor)` — возвращает текущее положение двигателя в шагах.
- `set_absolute_position(motor, position)` — устанавливает абсолютное положение двигателя в шагах.
- `set_relative_position(motor, steps)` — смещает двигатель на указанное число шагов.

- `get_speed(motor)` — возвращает текущую скорость двигателя в шагах в секунду.
- `set_speed(motor, speed)` — устанавливает скорость двигателя в шагах в секунду.
- `enable_motor(motor)` — включает питание двигателя.
- `disable_motor(motor)` — отключает питание двигателя.
- `get_status(motor)` — возвращает статус двигателя.
- `is_ready(motor)` — проверяет готовность двигателя, возвращает `True` если двигатель готов.

3.5 Управление затвором

- `set_shutter(number, open)` — управляет затвором, при значении `True` открывает, при `False` закрывает.
- `get_shutter(number)` — возвращает состояние затвора, `True` если открыт.

3.6 Сброс устройства

- `reset()` — выполняет полный сброс лазера.

3.7 Получение информации

- `get_model()` — возвращает модель лазера.
- `get_device_information()` — возвращает словарь с информацией об устройстве.
- `get_all_settings()` — возвращает словарь со всеми текущими настройками.

3.8 Константы модуля

- `STATUS_READY = 0` — двигатель готов к работе.
- `STATUS_BUSY = 1` — двигатель находится в движении.
- `STATUS_ERROR = -1` — произошла ошибка.

4 Модуль осциллографа

Управление цифровым осциллографом через интерфейс VISA.

4.1 Создание объекта

```

1 from oscilloscope import Oscilloscope
2
3 scope = Oscilloscope()
```

4.2 Подключение и отключение

- `connect(resource, timeout)` — подключается к осциллографу по указанному ресурсу, выполняет автоматический поиск если ресурс не указан, таймаут задаётся в миллисекундах.
- `disconnect()` — отключается от осциллографа.

4.3 Управление каналами

- `set_channel_enabled(channel, enable)` — включает или отключает указанный канал.
- `is_channel_enabled(channel)` — проверяет включён ли указанный канал, возвращает `True` если включён.
- `set_channel_scale(channel, scale)` — устанавливает вертикальный масштаб указанного канала в вольтах на деление.
- `get_channel_scale(channel)` — возвращает вертикальный масштаб указанного канала в вольтах на деление.
- `set_channel_offset(channel, offset)` — устанавливает вертикальное смещение указанного канала в вольтах.
- `get_channel_offset(channel)` — возвращает вертикальное смещение указанного канала в вольтах.
- `set_channel_coupling(channel, coupling)` — устанавливает тип связи указанного канала.
- `get_channel_coupling(channel)` — возвращает тип связи указанного канала.
- `set_channel_impedance(channel, impedance)` — устанавливает входное сопротивление указанного канала в омах.
- `set_channel_inverted(channel, invert)` — инвертирует сигнал на указанном канале.
- `set_channel_bandwidth_limit(channel, limit)` — ограничивает полосу пропускания указанного канала.

4.4 Управление временной развёрткой

- `set_timebase_scale(scale)` — устанавливает горизонтальный масштаб в секундах на деление.
- `get_timebase_scale()` — возвращает горизонтальный масштаб в секундах на деление.
- `set_timebase_delay(delay)` — устанавливает задержку развёртки в секундах.
- `set_timebase_mode(mode)` — устанавливает режим развёртки.

4.5 Управление триггером

- `set_trigger_source(source)` — устанавливает источник триггера.
- `set_trigger_level(level)` — устанавливает уровень триггера в вольтах.
- `set_trigger_slope(slope)` — устанавливает наклон триггера.
- `set_trigger_mode(mode)` — устанавливает режим триггера.
- `force_trigger()` — принудительно запускает триггер.
- `get_trigger_status()` — возвращает статус триггера.

4.6 Управление захватом

- `run_acquisition()` — запускает непрерывный захват сигнала.
- `stop_acquisition()` — останавливает захват сигнала.
- `single_acquisition()` — выполняет однократный захват сигнала.
- `is_acquisition_complete()` — проверяет завершение захвата, возвращает `True` если захват завершён.
- `set_acquisition_type(type)` — устанавливает режим усреднения.
- `set_average_count(count)` — устанавливает количество кадров для усреднения.
- `get_average_count()` — возвращает количество кадров для усреднения.

4.7 Захват данных

- `capture_waveform(channel, points)` — захватывает форму сигнала с указанного канала с заданным количеством точек, возвращает массивы времени и напряжения.
- `acquire_averaged_waveform(channel, average, points)` — захватывает усреднённый сигнал с указанного канала с заданным количеством точек и числом кадров усреднения.
- `acquire_averaged_waveform_retry(channel, average, points, retries)` — захватывает усреднённый сигнал с указанным числом попыток в случае ошибки.

4.8 Измерения сигнала

- `measure_voltage_peak_to_peak(channel)` — измеряет напряжение пик-пик на указанном канале в вольтах.
- `measure_frequency(channel)` — измеряет частоту сигнала на указанном канале в герцах.
- `measure_period(channel)` — измеряет период сигнала на указанном канале в секундах.

- `measure_rise_time(channel)` — измеряет время нарастания сигнала на указанном канале в секундах.
- `measure_fall_time(channel)` — измеряет время спада сигнала на указанном канале в секундах.
- `measure_duty_cycle(channel)` — измеряет скважность сигнала на указанном канале в процентах.
- `measure_delay(source1, source2)` — измеряет задержку между двумя указанными источниками.
- `measure_phase(source1, source2)` — измеряет фазовый сдвиг между двумя указанными источниками.
- `compute_waveform_statistics(values)` — вычисляет статистику сигнала: максимум, минимум, размах, среднее, среднеквадратичное значение и стандартное отклонение.

4.9 Сохранение данных

- `save_screenshot(filename)` — сохраняет скриншот экрана осциллографа в указанный файл.
- `save_setup(location)` — сохраняет текущие настройки в ячейку памяти по указанному номеру.
- `recall_setup(location)` — восстанавливает настройки из ячейки памяти по указанному номеру.

4.10 Получение информации

- `get_identification()` — возвращает строку идентификации осциллографа.
- `get_device_information()` — возвращает словарь с информацией об устройстве.
- `get_all_settings()` — возвращает словарь со всеми текущими настройками.

4.11 Константы модуля

- `COUPLING_AC = "AC"` — связь по переменному току.
- `COUPLING_DC = "DC"` — связь по постоянному току.
- `COUPLING_GND = "GND"` — заземление входа.
- `ACQ_NORMAL = "NORM"` — обычный режим усреднения.
- `ACQ_AVERAGE = "AVER"` — режим усреднения.
- `ACQ_PEAK = "PEAK"` — режим пикового детектора.
- `ACQ_HRES = "HRES"` — режим высокого разрешения.
- `SLOPE_POS = "POS"` — триггер по переднему фронту.

- `SLOPE_NEG = "NEG"` — триггер по заднему фронту.
- `SLOPE_EITH = "EITH"` — триггер по любому фронту.

5 Модуль энергометра

Управление измерителем мощности и энергии лазерного излучения.

5.1 Создание объекта

```
1 from energymeter import Energymeter
2
3 meter = Energymeter()
```

5.2 Подключение и отключение

- `connect(port, baudrate)` — подключается к энергометру через указанный порт и скорость, выполняет автоматический поиск если порт не указан.
- `disconnect()` — отключается от энергометра.
- `is_connected()` — проверяет наличие подключения, возвращает `True` при наличии соединения.

5.3 Измерения параметров

- `get_power()` — возвращает текущую мощность в ваттах.
- `get_energy()` — возвращает текущую энергию в джоулях.
- `get_average_power(count, delay)` — возвращает усреднённую мощность за указанное число измерений с заданной задержкой между ними.
- `get_average_energy(count, delay)` — возвращает усреднённую энергию за указанное число измерений с заданной задержкой между ними.

5.4 Управление шкалой

- `set_scale(index)` — устанавливает шкалу по индексу от нуля до сорока одного.
- `set_scale_up()` — увеличивает шкалу на одну позицию.
- `set_scale_down()` — уменьшает шкалу на одну позицию.
- `get_current_scale_index()` — возвращает текущий индекс шкалы.
- `set_autoscale(enable)` — включает автоматический выбор шкалы при значении `True` и отключает при `False`.
- `get_autoscale()` — проверяет включена ли автоматическая шкала, возвращает `True` если включена.

5.5 Настройка длины волны

- `set_wavelength_nanometers(wavelength)` — устанавливает длину волны в нанометрах.
- `set_wavelength_microns(wavelength)` — устанавливает длину волны в микрометрах.
- `get_wavelength()` — возвращает текущую длину волны в нанометрах.

5.6 Дополнительные настройки

- `set_trigger_level(level)` — устанавливает уровень триггера в процентах от нуля целых одной десятой до девяноста девяти целых девяти десятых.
- `get_trigger_level()` — возвращает уровень триггера в процентах.
- `set_noise_suppression(size)` — устанавливает подавление шума, принимает значения от нуля до девятисот девяноста девяти.
- `set_zero_offset()` — выполняет обнуление показаний.
- `clear_zero_offset()` — очищает обнуление показаний.
- `set_attenuator(enable)` — включает аттенюатор при значении `True` и отключает при `False`.
- `get_attenuator()` — проверяет состояние аттенюатора, возвращает `True` если включён.
- `set_user_multiplier(multiplier)` — устанавливает пользовательский множитель.
- `get_user_multiplier()` — возвращает пользовательский множитель.
- `set_user_offset(offset)` — устанавливает пользовательское смещение.
- `get_user_offset()` — возвращает пользовательское смещение.
- `set_baud_rate(index)` — устанавливает скорость порта по индексу от нуля до четырёх.

5.7 Получение информации

- `get_version()` — возвращает версию прошивки устройства.
- `get_status()` — возвращает статус устройства.
- `get_extended_status()` — возвращает расширенный статус устройства.
- `get_device_information()` — возвращает словарь с информацией об устройстве.
- `get_all_settings()` — возвращает словарь со всеми текущими настройками.

5.8 Константы модуля

- `MODE_POWER = 0` — режим измерения мощности.
- `MODE_ENERGY = 1` — режим измерения энергии.
- `BAUD_9600 = 0` — скорость порта 9600 бод.
- `BAUD_19200 = 1` — скорость порта 19200 бод.
- `BAUD_38400 = 2` — скорость порта 38400 бод.
- `BAUD_57600 = 3` — скорость порта 57600 бод.
- `BAUD_115200 = 4` — скорость порта 115200 бод.

6 Практические примеры

6.1 Подключение всех устройств

```
1 from chromator import Chromator
2 from laser_source import LaserSource
3 from oscilloscope import Oscilloscope
4 from energymeter import Energymeter
5
6 # Создание объектов
7 chromator = Chromator()
8 laser = LaserSource()
9 scope = Oscilloscope()
10 meter = Energymeter()
11
12 # Подключение
13 chromator.connect()
14 laser.connect()
15 scope.connect()
16 meter.connect()
```

6.2 Проведение измерения

```
1 # Настройка длины волны
2 chromator.set_wavelength(500.0)
3 laser.set_wavelength(500.0)
4
5 # Настройка осциллографа
6 scope.set_channel_enabled(1, True)
7 scope.set_channel_scale(1, 1.0)
8 scope.run_acquisition()
9
10 # Захват сигнала
11 time_values, volt_values = scope.capture_waveform(1, 2000)
12 power = meter.get_power()
```

6.3 Сохранение результатов

```
1 # Сохранение в CSV
2 with open("data.csv", "w") as new_file:
3     new_file.write("time_seconds,voltage_volts\n")
4     for time, value in zip(time_values, volt_values):
5         new_file.write(f"{time:.8e},{value:.6f}\n")
6
7 print(f"Мощность: {power:.6e} Вт")
```

6.4 Завершение работы

```
1 # Отключение устройств
2 chromator.disconnect()
3 laser.disconnect()
4 scope.disconnect()
5 meter.disconnect()
```